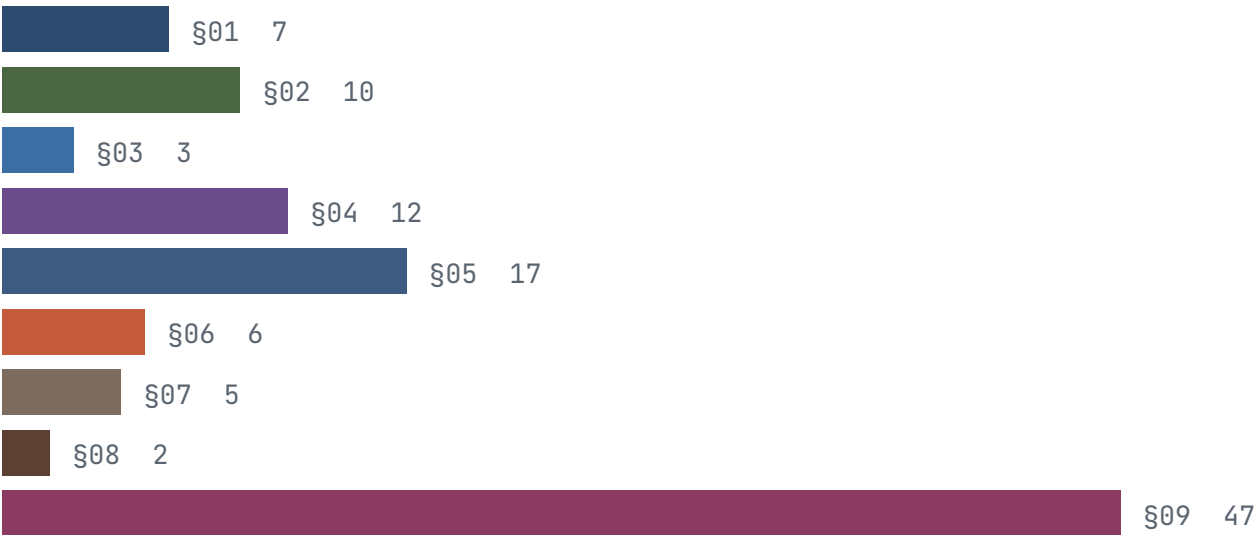


# System Atlas

Cursor グローバルスキル・カタログ

SPECIMEN COUNT: 109    ·    CATEGORIES: 9    ·    2026-10-07



凡例 — コマンド / 説明 / 使いどころ

■ 手動のみ（明示呼び出し）

# INDEX

9カテゴリ・縦断カタログ — 各章は独立 folio

§01

オーケストレーター

7 skills

§02

ドキュメント／データ処理

10 skills

§03

Google Workspace 連

3 skills

§04

設計・開発ワークフロー

12 skills

§05

独自の開発系スキル

17 skills

§06

デザイン・コンテンツ制作

6 skills

§07

ナレッジ管理・リサーチ

5 skills

§08

GitHub / Git 運用

2 skills

§09

マーケティング

47 skills

- ・ 運用: ~/.cursor/skills/ にグローバル配置
- ・ Playbook = 複数スキルの進行台本（手動のみ）
- ・ マーケ46件 = product-marketing.md を先に確認

# PLAYBOOK ROUTES

進行台本 — 矢印は推奨実行順序

## /playbook-app-improvement

systematic-debugging → improve-codebase-architecture → requesting-code-review → react  
再発バグ、複雑化した既存アプリ



## /playbook-article-production

content-research-writer → doc-coauthoring → edit-article → \*\*marketing \*\*  
X投稿・note記事・メルマガ・セミナー告知文



## /playbook-document-data

pdf/docx/xlsx → Phase 1 → gws → Phase 2  
見積・請求・契約の整理、Excel/Word 比較表、Drive 資料



## /playbook-fable5-7day

skill-creator → brainstorming → grill-me → verification-before-completion → systemati  
自分の業務 OS を作り直したい／スキル基盤を整えたい



## /playbook-mini-webapp

to-prd → frontend-design → web-artifacts-builder → webapp-testing → verification-befo  
社内管理表・フォーム・簡易ダッシュボード



## /playbook-research-assets

notebooklm → obsidian-vault markdown / canvas → content-research-writer  
散在ナレッジの資産化



## /route-playbook

/playbook-\* 1 GO  
「どのplaybook?」「台本を選んで」「記事書いて／見積まとめて」など入口が曖昧なとき

## §01 オーケストレーター（playbooks・7件）

手動のみ。日常パックに含まれるのはルーター1＋台本6。中身のスキル自体は自動で連鎖しない。

### §01.01

#### /playbook-app-improvement

systematic-debugging → improve-codebase-architecture → requesting-code-review → react-best-practices（非 React はスキップ可）。  
再発バグ、複雑化した既存アプリ

### §01.02

#### /playbook-article-production

content-research-writer → doc-coauthoring → edit-article →（任意・\*\*marketing パック導入時のみ\*\*訴求調整）。  
X投稿・note記事・メルマガ・セミナー告知文

### §01.03

#### /playbook-document-data

pdf/docx/xlsx 取込・構造化（Phase 1）→ 任意で gws（Phase 2）。\*\*Firecrawl は明示指示がない限り使わない。\*\*  
見積・請求・契約の整理、Excel/Word 比較表、Drive 資料

### §01.04

#### /playbook-fable5-7day

skill-creator → brainstorming → grill-me → verification-before-completion → systematic-debugging。業務棚卸しからスキル作成・設計・穴埋め・QA・デバッグ。  
自分の業務 OS を作り直したい／スキル基盤を整えたい

### §01.05

#### /playbook-mini-webapp

to-prd → frontend-design → web-artifacts-builder → webapp-testing → verification-before-completion。  
社内管理表・フォーム・簡易ダッシュボード

§01.06

**/playbook-research-assets**

notebooklm → obsidian-vault (+ 任意で markdown / canvas) → content-research-writer。  
散在ナレッジの資産化

§01.07

**/route-playbook**

要求を見て最適な /playbook-\* を1つ (+ 代替) 提案し、適応スケッチを出して GO 待ち。実行はしない。マーケ未導入なら playbook-lp-creative は候補から外すか導入を促す。  
「どのplaybook?」「台本を選んで」「記事書いて／見積まとめて」など入口が曖昧なとき

## §02 ドキュメント／データ処理（10件）

### §02.01

#### **/doc-coauthoring**

文書・提案書・技術仕様・意思決定文書を「文脈収集→推敲・構造化→読者テスト」の3段階で共同執筆。

「ドキュメントを書きたい」「提案書を作りたい」「PRDを書きたい」など、ある程度分量のある文章作成

### §02.02

#### **/doc-maint**

既存リポジトリの README / docs を読み取り専用監査し、維持・統合・修正・移動・要確認の5分類で提案。承認後に文書だけ適用。コードは触らない。project-foundation が既存と判定したときの受け先。

「READMEを直して」「docsを整理して」「ドキュメントが古い」「重複を消して」「オンボーディングを分かりやすく」

### §02.03

#### **/docx**

Word 文書（.docx）の作成・読取・編集。目次・見出し・ページ番号・レターヘッド付き文書、画像挿入・置換、検索置換、変更履歴・コメント対応。

「Wordドキュメント」「.docx」「レポート」「メモ」「レター」「テンプレート」の作成・編集依頼

### §02.04

MANUAL

#### **/improve-codebase-architecture**

コードベースをスキャンして「深化の余地」をHTMLレポートで可視化し、選んだ項目についてgrill-me形式で深掘り。

過去のコードの複雑化を解消したい、リファクタリング候補を洗い出したい

### §02.05

#### **/pdf**

PDF のテキスト・表抽出、結合・分割、回転、透かし、新規作成、フォーム入力、暗号化/復号、画像抽出、スキャンPDFのOCR。

「.pdf」に言及、または PDF を作りたい／読みたい／編集したいとき

## §02.06

**/pptx**

.pptx が入出力どちらかに関わる全作業。スライド作成、テキスト抽出、既存プレゼン編集、テンプレート/レイアウト/スピーカーノート操作。

「デッキ」「スライド」「プレゼン」「.pptx」に言及されたら常に使う

## §02.07

**/promote-skill**

スキル作成・編集の直後に、incoming の4箱のどれかへ置き、scripts/sync-incoming.ps1 で pack・一覧・グローバルへ反映する。手で ~/.agents / ~/.cursor / skills-pack へコピーしない。Cursor・Claude Code 両対応。パス未検出時は聞いて、検証前は書かない。

create-skill / skill-creator の完了後、「同期して」「skills-pack に入れる」

## §02.08

**/skill-creator**

新規スキル作成、既存スキルの編集・最適化、評価(eval)実行、性能ベンチマーク。

スキルを新規作成したい／SKILL.mdの書き方を聞かれたとき

## §02.09

**/theme-factory**

スライド・文書・レポート・LP等に一貫したテーマ（色・フォント）を適用。10種のプリセットテーマ or 新規生成。

成果物のトンマナを統一したいとき。marketing パックの `playbook-lp-creative` でも使用

## §02.10

**/xlsx**

既存/新規の .xlsx・.xlsm・.csv・.tsv の読取・編集・修正（列追加、数式計算、書式設定、グラフ化、汚いデータの整形）。成果物は必ずスプレッドシート。

「このExcelを直して」「スプレッドシートを作って」など、成果物がスプレッドシートである依頼

## §03 Google Workspace 連携（3件）

前提: `gws` CLI (`@googleworkspace/cli`) のインストールと OAuth 認証 (`gws auth login`) が別途必要

### §03.01

#### **/gws-docs**

Google Docs の読み書き（gws docs コマンドのラッパー）。  
Google ドキュメントの内容を読む・編集する依頼

### §03.02

#### **/gws-drive**

Google Drive のファイル・フォルダ・共有ドライブ管理（gws drive コマンドのラッパー）。  
Drive 上のファイル検索・一覧・整理の依頼

### §03.03

#### **/gws-sheets**

Google Sheets の読み書き（gws sheets コマンドのラッパー）。  
Google スプレッドシートの内容を読む・編集する依頼

## §04 設計・開発ワークフロー（Superpowers系・12件）

### §04.01

#### **/brainstorming**

創造的作業（機能追加・コンポーネント作成・挙動変更）の\*\*前に必ず\*\*要件と設計を対話的に探索。承認なしに実装させない強いゲート。

何か新しく作る・変更する前段階。「これは簡単すぎて設計不要」という思い込みへの対抗策

### §04.02

#### **/dispatching-parallel-agents**

2つ以上の独立したタスク（別ファイル・別バグ等）を並列サブエージェントで同時処理。

複数の無関係な失敗やタスクを同時に片付けたいとき

### §04.03

#### **/executing-plans**

書かれた実装プランを読み込み、批判的にレビューしてからチェックポイント付きでタスクを実行。

すでにプランファイルがあり、別セッションで実行したいとき

### §04.04

#### **/finishing-a-development-branch**

実装完了・テスト通過後、マージ/PR/破棄などの選択肢を整理して提示し、選んだワークフローを実行。

開発ブランチの後始末をどうするか決めるとき

### §04.05

#### **/receiving-code-review**

レビュー指摘を鵜呑みにせず、技術的に検証してから対応。読む→理解→検証→評価→応答→実装の順で処理。

コードレビューのフィードバックを受け取ったとき

## §04.06

**/requesting-code-review**

完了したタスク・大きな機能実装後・マージ前に、専用コンテキストを持つレビューアサブエージェントへレビュー依頼。

タスク完了時、マージ前、行き詰まったときの新しい視点

## §04.07

**/subagent-driven-development**

タスクごとに新規サブエージェントを起動し、実装後に毎回レビュー、最後に全体レビュー。独立したタスクが並ぶ実装プランを、今のセッション内で高品質に進めたいとき

## §04.08

**/systematic-debugging**

バグ・テスト失敗・想定外挙動に遭遇したら、\*\*必ず根本原因調査を先に完了してから\*\*修正提案する4段階プロセス。

あらゆるバグ調査の前段階。「とりあえず直す」を禁止

## §04.09

**/using-git-worktrees**

既存の隔離状態を検知した上で、feature作業やプラン実行前に隔離ワークスペース（git worktree等）を用意。

設計承認後、本体を汚さず並行作業したいとき

## §04.10

**/using-superpowers**

スキルシステムの使い方ブートストラップ。「1%でも関連しそうならスキルを必ず使う」という原則を強制。

セッション開始時（自動判定用の基盤スキル）

## §04.11

**/verification-before-completion**

「完了した」「直った」と主張する前に、検証コマンドを実際に実行し出力で裏付ける。証拠なき完了宣言を禁止。

完了報告・コミット・PR作成の直前

§04.12

### **/writing-skills**

新規スキルの作成・編集・デプロイ前の検証。TDD（RED-GREEN-REFACTOR）をドキュメント作成に応用。

新しいスキルを書く、既存スキルを直す、公開前の動作確認

## §05 独自の開発系スキル (17件)

§05.01

MANUAL

### **/00**

メッセージ先頭の /00 で、そのターンだけ using-superpowers と Superpowers 由来の必須ワークフロー（計画・TDD・デバッグ・レビュー・検証・サブエージェント・worktree）を無効化。依頼本文は通常どおり処理。次のメッセージへは引き継がない。

Superpowers を使わず、今の依頼だけ素早く処理したいとき。文章中やコード内の `/00` では発動しない

§05.02

MANUAL

### **/chat-handoff**

別PCで続きをやるための引き継ぎメモを、リポジトリ直下の 質疑応答M-D.MD に作成・追記。結論・決定・未解決TODOを残す。Cursor チャット履歴は端末に残る前提。

「別PC用に残して」「引き継ぎ書いて」「質疑応答に落として」「続きは別端末で」と明示されたとき。通常の備忘録は `/session-recap`

§05.03

### **/git-guardrails**

危険なGitコマンド（push、reset --hard、clean、branch -D等）を実行前にブロックするフックを設定。

誤操作防止のガードレールを設定したいとき

§05.04

### **/model-router-gpt**

CodexのGPTモデルへ、大量の原文を読む調査を渡す

リポジトリ全体や大量の文書・ログから根拠を探すとき。少数ファイルの確認には使わない

§05.05

MANUAL

### **/new-project**

空ディレクトリ向け scaffold。CLI 対話。必須は README。Hidem プロファイルは選択式。既存ファイルは上書きしない。

`project-foundation` が新規と判定したとき。既存リポジトリでは使わない

## §05.06

**/project-foundation**

新規/既存/判定不能を振り分ける文書標準の入口。判定中は書き込まない。new → new-project、existing → doc-maint。

「新規プロジェクト」「scaffold」「READMEを直して」「docsを整理して」

## §05.07

**/react-best-practices**

Vercel Engineering によるReact/Next.jsパフォーマンス最適化ガイド（8カテゴリ70ルール）

。React/Next.jsのコンポーネント作成・データフェッチ実装・パフォーマンスレビュー・リファクタリング

## §05.08

MANUAL

**/session-recap**

今の会話を 備忘録.md の日付エントリにする（決定・調査・次のTODO）。コード変更履歴用の 変更履歴.md には書かない。

「まとめて」「備忘録に残して」「このやり取りを記録して」と明示されたときだけ

## §05.09

**/test-driven-development**

RED-GREEN-REFACTORのTDDを実装前に徹底。ローカル改良版（Superpowers本体+マルチスタックテストコマンド表付き）。

機能追加・バグ修正・リファクタリング前。「TDDで」「テストファーストで」と言われたとき

## §05.10

MANUAL

**/to-issues**

プラン・仕様・PRDを、独立して着手可能な「トレーサーバレット」の垂直スライスIssueに分解。

PRDや設計書を実装可能な単位のIssueに分割したいとき

## §05.11

MANUAL

**/to-knowledge**

今の会話から、残す価値のある決定・失敗・再現コストの高い設定だけを拾い、Obsidian の D: ¥vault¥04\_知見¥へ提案する。書く前に承認を取る。作業ログは書かない。

「知見に落として」「今日の分を知見化して」「残す価値あるもの拾って」と明示されたとき。時系列の備忘録は `~/session-recap`

## §05.12

MANUAL

**/to-prd**

今の会話をヒアリングなしでそのままPRDに合成し、Issueトラッカーへ公開。  
すでに会話で要件が固まっており、そのままPRD化したいとき

## §05.13

**/web-artifacts-builder**

React 18 + TypeScript + Vite + Tailwind + shadcn/ui で、状態管理やルーティングを含む多  
コンポーネントのHTMLアーティファクトを構築。  
単純な単一ファイルHTML/JSXでは足りない、複雑なWebアーティファクトを作りたいとき

## §05.14

**/webapp-testing**

Cursorのブラウザ MCP または Playwright スクリプトでローカル Web アプリをテスト。ユニ  
ット/E2E(IDE)/E2E(スクリプト)の使い分け表あり。  
「アプリをテストして」「UIを確認して」、フロントエンドの動作検証全般

## §05.15

**/writing-plans**

ファイルパス・コード断片・検証手順まで含めた実装プランを、コーディング前に作成。ロー  
カル Cursor 向け Adapt 版。  
仕様が複数ステップにわたる非自明な機能に着手する前

## §05.16

**/x-reader**

内蔵ブラウザで x.com / twitter.com の投稿本文と添付を読む  
x.com または twitter.com の投稿URLを読んで、開いて、確認して、要約してほしいとき

## §05.17

**/youtube-music-playlist**

曲リストのURLから再生リストへ追加し、曖昧な曲は保留して保存先で確認する  
渡された曲リストやYouTube Musicのリンクを、ログイン済みブラウザの再生リストへ追加・再開する  
とき

## §06 デザイン・コンテンツ制作（6件）

### §06.01

#### **/canvas-design**

.png/.pdfで独自のビジュアルアート（ポスター・作品・デザイン）を、デザイン哲学の作成→視覚表現の2段階で制作。

ポスター、アート作品、静的なビジュアルデザインを作りたいとき

### §06.02

#### **/content-research-writer**

リサーチ・引用付与・フック改善・アウトライン反復・セクションごとのリアルタイムFBで、執筆をパートナー作業に変える。

ブログ・記事・ニュースレター・技術文書の執筆、根拠付きの文章を書きたいとき

### §06.03

MANUAL

#### **/debug-allrun**

デバッガ付きで起動し、サイレントなデータ破損・非同期レース・分岐ミスという3種の隠れバグを能動的に探索・自動修正・報告。

`/debug-allrun` 明示呼び出し時。「クラッシュしないから安全」を信用しない徹底デバッグ

### §06.04

MANUAL

#### **/edit-article**

記事を見出しで章分けし、各章を明確さ・一貫性・流れの観点で書き直す（1段落最大240字）

。記事の下書きを編集・改善・推敲したいとき

### §06.05

#### **/frontend-design**

テンプレート感のない、意図を持った視覚デザインの方向性（配色・タイポグラフィ・レイアウト）を設計思想ガイド。

新規UI構築、既存UIの作り直し、「他と被らないデザインにしたい」とき

§06.06

**/grill-me**

プラン・設計について執拗に一問一答で質問し、決定木の分岐を一つずつ潰して共通理解に至る。各質問に推奨回答も提示。

「grill me」「プランを詰めて」と言われたとき、設計の穴を潰したいとき

## §07.01

**/app-tech-inventory**

依存関係ファイルから技術スタックを YAML に残し、本文に何のアプリか・できること・直近の変化（タグ間、無ければ90日、最大7行）を書く。未反映は全件保存後にまとめて doc-maint（README限定）を提案。

「このアプリ何のフレームワーク使ったっけ」「何のアプリ？何ができる？」、アプリ台帳の棚卸し、複数リポジトリの一括確認

## §07.02

**/json-canvas**

.canvas ファイル（ノード・エッジ・グループ）の生成・編集。D:¥vault¥canvas¥ 推奨。kepano/obsidian-skills 由来。

マインドマップ、フローチャート、Index の視覚俯瞰を Canvas で作りたいとき

## §07.03

**/notebooklm**

Google NotebookLM のノートブックにブラウザ自動化で問い合わせ、Geminiの根拠付き回答を取得。ハルシネーションを大幅抑制。

NotebookLMのURLを共有された、「自分のNotebookLMに聞いて」と言われたとき

## §07.04

**/obsidian-markdown**

callout、frontmatter (properties)、embed、wikilink 等の Obsidian Flavored Markdown 文法。kepano/obsidian-skills 由来。

Obsidian ノートの書式・callout・メタデータを正確に書きたいとき

## §07.05

**/obsidian-vault**

D:¥vault 内のノートを wikilink・インデックスノートで検索・作成・整理。vault パスと命名規則の正本。

Obsidianでノートを探す・作る・整理したいとき

## §08.01

MANUAL

**/git-in-clone**

指定したGitHubリポジトリを現在の空ディレクトリへ `git clone .` で取得。  
「このフォルダにクローンして」と言われたとき

## §08.02

MANUAL

**/github-make-sync**

指定した名前で非公開GitHubリポジトリを作成し、gh CLIで現在のプロジェクトに `origin` を設定。  
新規プロジェクトをGitHubに接続したいとき

## 入口 playbook

§09.01

**/playbook-lp-creative**

marketingskills（訴求設計）→ frontend-design → canvas-design → theme-factory → pptx。  
Adaptive proposal gate 付き。  
商品ローンチ、無料特典、セミナー募集の制作一式（\*\*パック導入後\*\*）

## 戦略・計画系

§09.02

**/marketing-ideas**

139件の実証済みマーケティング施策ライブラリから、状況・ステージ・リソースに合う施策を提案。  
「何をすればいいかわからない」「マーケティングのアイデアが欲しい」とき

§09.03

**/marketing-loops**

週次SEOスキャン、広告疲労チェック、解約シグナル監視など、AIエージェントが一定周期で回す「マーケティングループ」の設計・スケジューリング。  
一度きりでなく継続的に回る仕組みを作りたいとき

§09.04

**/marketing-plan**

fCMOレベルの、AARRR構造・139のアイデアライブラリ・17項目監査ループリックを組み込んだ包括的な12ヶ月マーケティングプランを生成。  
クライアントや自社の本格的な成長計画・GTMプランが欲しいとき

§09.05

**/marketing-psychology**

アンカリング・社会的証明・希少性・損失回避などの心理原則・メンタルモデルをマーケティングに応用。  
「なぜ人は買うのか」を踏まえた施策・訴求を考えたいとき

§09.06

### **/product-marketing**

プロダクトマーケティングの共通コンテキスト文書（.agents/product-marketing.md）を作成・更新。他の全マーケティングスキルが参照する起点。

新規プロジェクトでマーケティング系スキルを使う前に最初にやる

## コピー・コンテンツ系

§09.07

### **/content-strategy**

何を書くべきかのトピック選定・コンテンツピラー・エディトリアルカレンダー計画。

「何を書けばいいかわからない」ときの戦略レイヤー

§09.08

### **/copy-editing**

既存コピーの編集・レビュー・改善（リライトではなく強化）。1回のパスで1観点に集中。

すでにあるコピーを磨きたい、古くなったコンテンツを更新したいとき

§09.09

### **/copywriting**

ホームページ・LP・料金・機能ページ等のマーケティングコピーを新規作成・改善。

「コピーを書いて」「もっと魅力的に」と言われたとき

## 広告・クリエイティブ系

§09.10

### **/ad-creative**

見出し・説明文・本文などの広告クリエイティブを量産・反復生成。

「広告文のバリエーションが欲しい」とき

§09.11

### **/ads**

Google/Meta/LinkedIn/X等の有料広告キャンペーン戦略・ターゲティング・入札・最適化。

広告キャンペーンの立ち上げ・改善相談

§09.12

### **/image**

ブログヒーロー・SNS画像・製品モックアップ等のマーケティング画像をAI生成・最適化。  
マーケティング用の画像を作りたいとき

§09.13

### **/social**

LinkedIn/X/Instagram/TikTok等のSNSコンテンツ作成・スケジューリング・ソーシャルリスニング。  
SNS投稿の作成・トレンド監視・エンゲージメント戦略

§09.14

### **/video**

AIツールやプログラマティックフレームワークでマーケティング動画を制作。  
製品デモ、説明動画、SNS動画を作りたいとき

## コンバージョン最適化系

§09.15

### **/cro**

LP・料金ページ・フォーム等のコンバージョン率最適化分析と改善提案。  
「このページのCVRが低い」「改善してほしい」とき

§09.16

### **/offers**

オファー自体の設計（価値訴求、ボーナス、保証、緊急性、命名、支払い構造）。  
サービス・コース・高額商材のオファー作り

§09.17

### **/onboarding**

サインアップ後のオンボーディング・アクティベーション・初回体験の最適化。  
「登録はされるが使われない」とき

§09.18

### **/paywalls**

アプリ内ペイウォール・アップグレード画面・アップセルモーダルの設計最適化。  
無料→有料転換の画面を作りたいとき

§09.19

### **/popups**

ポップアップ・モーダル・スライドイン・バナーの設計最適化。  
メール獲得ポップアップ、離脱防止オーバーレイなど

§09.20

### **/pricing**

SaaS料金・パッケージング・マネタイズ戦略設計。  
料金プランの設計・見直し相談

§09.21

### **/signup**

サインアップ・登録・トライアル開始フローの離脱削減・完了率向上。  
登録フローがうまく機能していないとき

## **メール・SMS系**

§09.22

### **/cold-email**

返信率を高めるB2Bコールドメールとフォローアップ連続文の作成。  
新規開拓・営業メールを書きたいとき

§09.23

### **/emails**

ドリップ・ナーチャリング・ライフサイクルなど自動化メールシーケンス設計。  
ウェルカムメール、再エンゲージメール等の設計

§09.24

### **/sms**

ウェルカム・カート放棄・購入後・ウィンバック等のSMS/MMSマーケティング設計。  
SMS施策を計画・構築したいとき

## **分析・実験系**

§09.25

### **/ab-testing**

A/Bテスト・実験の設計、実験バックログ運用。  
「どちらのバージョンが良いか検証したい」とき

§09.26

### **/analytics**

GA4等の計測・トラッキング実装の設定・改善・監査。  
「イベントが計測できているか確認したい」とき

§09.27

### **/competitor-profiling**

競合URLリストから構造化された競合プロファイルを作成。  
複数の競合を深く調査したいとき

§09.28

### **/customer-research**

顧客インタビュー・レビュー・チケット分析からインサイトを抽出、または新規に外部調査。  
顧客の声を分析したい、ペルソナを作りたいとき

## **SEO・発見性系**

§09.29

### **/ai-seo**

ChatGPT/Perplexity/AI Overviews等のAI検索・AI回答内で引用されるための最適化。  
AI検索エンジンでの露出を高めたいとき

§09.30

### **/aso**

App Store / Google Playのアプリストア最適化診断。  
アプリのストア掲載を改善したいとき

§09.31

### **/directory-submissions**

スタートアップ/SaaS/AI向けディレクトリへの登録戦略（被リンク・発見性）。  
Product Hunt等のディレクトリ登録を計画したいとき

§09.32

### **/programmatic-seo**

テンプレートとデータでSEO用ページを大量生成。  
都市名×キーワード等、パターン化されたページを量産したいとき

§09.33

### **/schema**

schema.org構造化データ・JSON-LDの実装。  
リッチリザルト表示のための構造化データ追加

§09.34

### **/seo-audit**

サイトのSEO問題の診断・技術的SEOレビュー。  
「ランキングが落ちた」「SEOを見てほしい」とき（曖昧な相談でもまずこれ）

§09.35

### **/site-architecture**

サイトのページ階層・ナビゲーション・URL構造・内部リンクの計画。  
サイト全体の構成を設計・見直ししたいとき

## **獲得・アウトバウンド系**

§09.36

### **/co-marketing**

共同マーケティングパートナーの発掘・共同キャンペーン企画。  
他社との協業施策を考えたいとき

§09.37

### **/community-marketing**

Discord/Slackコミュニティの立ち上げ・成長戦略。  
コミュニティ主導成長を仕掛けたいとき

§09.38

### **/free-tools**

リード獲得・SEO・認知向上のための無料ツール企画。  
無料計算機・診断ツール等で集客したいとき

§09.39

### **/launch**

プロダクトローンチ・機能発表・リリース戦略。  
新機能・新商品の発表を計画したいとき

§09.40

### **/lead-magnets**

メール獲得のためのダウンロード資料（ebook、チェックリスト等）企画。  
リードマグネットを作りたいとき

§09.41

### **/prospecting**

B2B SaaS/一般B2B/地域中小企業向けの見込み客リスト構築・選定。  
アウトバウンド用のリードリストを作りたいとき

§09.42

### **/public-relations**

プレスリリース・記者アウトリーチ・アーンドメディア戦略。  
メディア掲載を狙いたいとき

§09.43

### **/referrals**

紹介プログラム・アフィリエイトプログラムの設計最適化。  
既存顧客からの紹介を増やす仕組みを作りたいとき

## 収益・営業系

§09.44

### **/churn-prevention**

解約防止フロー・セーブオファー・支払い失敗の回収施策設計。  
解約率が高い、キャンセルフローを作りたいとき

§09.45

### **/competitors**

競合比較ページ・オルタナティブページの作成（SEO＋営業支援）。  
「vs競合」「～の代替」ページを作りたいとき

§09.46

### **/revops**

リードスコアリング・ルーティング・営業引き渡しプロセスの設計。  
マーケ→営業の引き渡しが機能していないとき

§09.47

### **/sales-enablement**

営業デッキ・1ページ資料・反論対応集・デモ台本の作成。  
営業チーム向け資料を作りたいとき

# Colophon

出典: skills一覧.md · skills-pack/MANIFEST.json

配布: skills-pack/ 一式 → ~/.cursor/skills/

重複回避: skills-pack/skills重複処理.md

GWS連携: PCごとに gws auth login が必要

skills-cursor/ は本カタログ対象外